CUET PG

DAILY CLASS NOTES

Computer Science

Digital Fundamental

Lecture – 15

Registers



Registers

Content: Introduction to Registers, Categories of Registers, Properties of Registers.

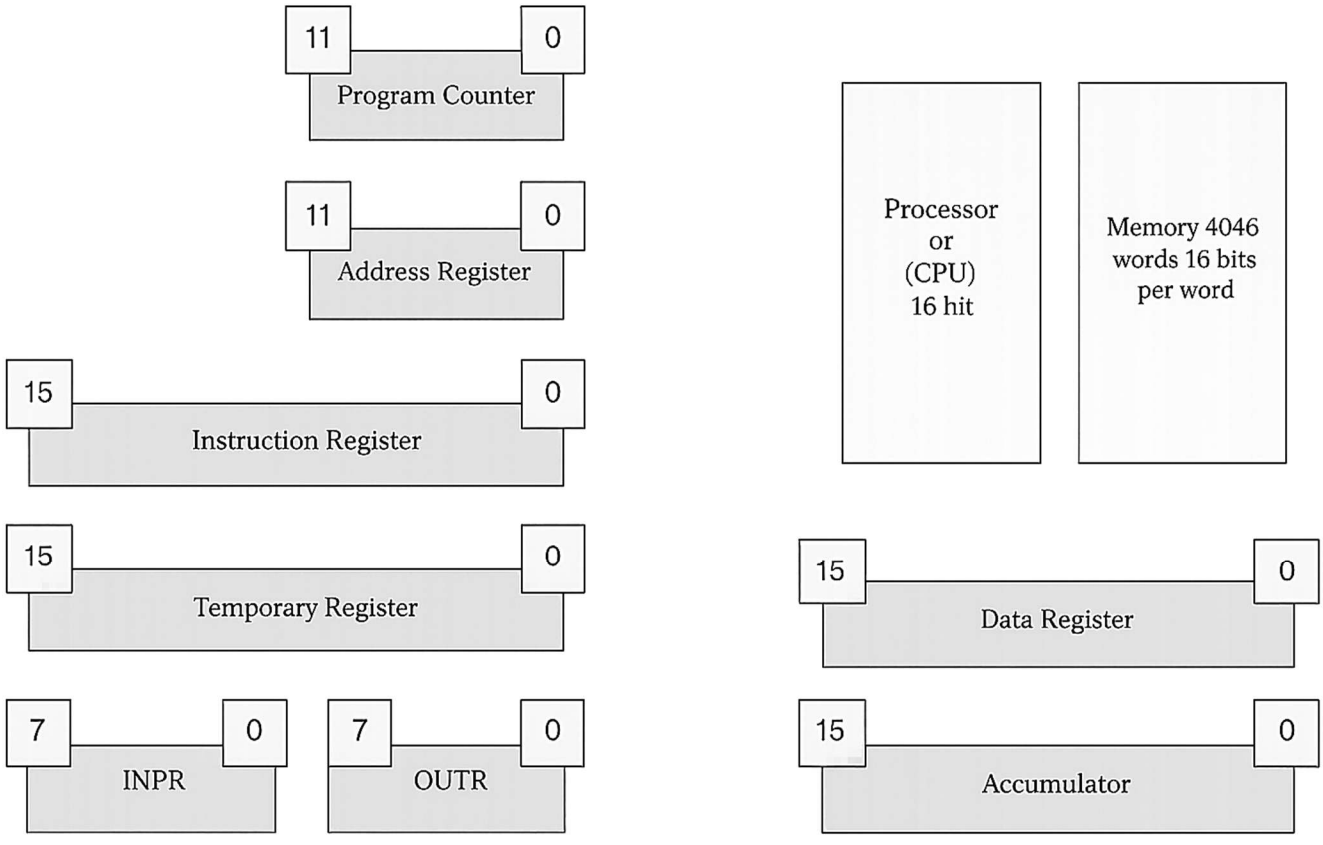
What is a Register in a Computer?

- ★ A register is a very small and very fast storage unit inside the CPU.
- ★ It stores data, instructions, or addresses for a short time while the CPU is working.
- ★ Registers are the fastest type of memory in a computer.
- ★ They help the CPU perform operations quickly because the CPU can access them instantly.
- ★ A register can store:
  - Bits (0s and 1s)
  - Instructions
  - Memory addresses
  - Any small amount of data is needed for quick processing

Why Registers Are Important

- ★ **Registers are faster than cache and RAM**, so the CPU can get data instantly without waiting.
- ★ They **reduce memory access time** because important data stays inside the CPU itself.
- ★ Registers **store intermediate results** (temporary answers) and **final results** during calculations.
- ★ They are used for **executing instructions**, **manipulating data**, and **keeping track of CPU status** (like flags, program counter, etc.).

Category/ Classes of CPU Register:



1. **General Purpose Registers (GPRs)**
   - ★ Symbol: R0, R1, R2, ..., Rn
   - ★ Used to store temporary data, intermediate results, and values during program execution.
   - ★ Accessed by assembly-level instructions.
   - ★ Widely used in register-to-register operations (faster than memory-based operations).

2. **Accumulator Register (AC)**
   - ★ Stores intermediate arithmetic and logic results from the ALU.
   - ★ Acts as a default operand/result location if not explicitly mentioned in the instruction.
   - ★ Frequently used in older or simpler CPUs.

3. **Program Counter (PC)**
   - ★ Holds the address of the next instruction to be executed.
   - ★ Automatically incremented after each fetch cycle.
   - ★ Size depends on architecture (e.g., a 12-bit PC can address 4K memory locations).

4. **Instruction Register (IR)**
   - ★ Holds the current instruction fetched from memory.
   - ★ The instruction is decoded and executed from here.
   - ★ Number of bits = number of bits in an instruction.

5. **Memory Address Register (MAR)**
   - ★ Holds the memory address of the data or instruction to be fetched or stored.
   - ★ Used alongside MDR for memory operations.

6. **Memory Data Register (MDR) / Memory Buffer Register (MBR)**
   - ★ Temporarily stores data being transferred to/from memory.
   - ★ Acts as a data interface between memory and the CPU.

7. **Stack Pointer (SP)**
   - ★ Points to the top of the stack in memory.
   - ★ Stack is used for function calls, interrupts, and recursive procedures.
   - ★ SP updates automatically on PUSH and POP.

8. **Flag Register / Status Register / Condition Code Register (CCR)**
   - ★ Stores binary flags that describe the outcome of operations.
   - ★ **Common flags:**
     - ➢ Carry (C): Set if there is a carry/borrow.
     - ➢ Zero (Z): Set if the result is 0.
     - ➢ Sign (S) or Negative (N): Set if the result is negative.
     - ➢ Overflow (V): Set on signed overflow.
     - ➢ Parity, Auxiliary Carry, Interrupt Enable, etc.
     - ➢ Flags are used in branching and conditional logic.



9. Temporary Register (TR)

- ★ Holds temporary intermediate data during complex CPU operations.
- ★ Not programmer-accessible, used internally by the control unit.

10. Input Register (INPR) and Output Register (OUTR)

- ★ INPR stores characters from input devices.
- ★ OUTR holds characters to be sent to output devices.
- ★ Typically, 8-bit registers depend on character encoding.

Common Registers and Their Properties

Register Table

Register Name	Symbol	Bits	Function Description
Accumulator	AC	16	Stores the result of arithmetic/logic operations
Program Counter	PC	12	Holds the address of the next instruction
Address Register	AR	12	Holds the address of data/instruction
Data Register	DR	16	Holds operand/data fetched from memory
Instruction Register	IR	16	Stores the instruction currently being executed
Temporary Register	TR	16	Stores temporary values during processing
Input Register	INPR	8	Holds input character from the input device
Output Register	OUTR	8	Holds output character for the output device

How Registers Work with Other CPU Components

Registers communicate with:

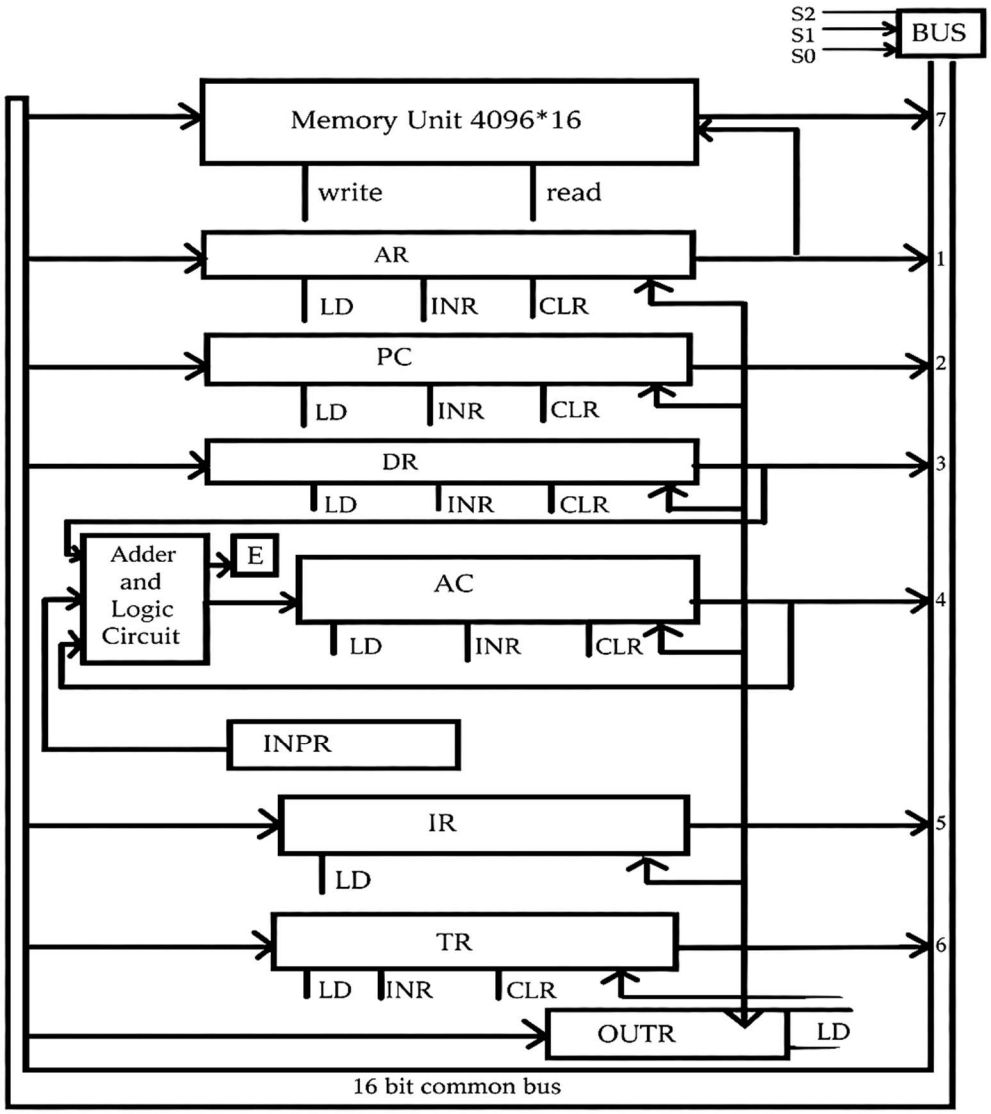
- ★ ALU for arithmetic and logical operations
- ★ Control Unit for instruction execution control
- ★ Memory for data/instruction fetch and storage
- ★ I/O devices for data transfer (INPR/OUTR)
- ★ Registers enable fast access and processing, reducing reliance on slower RAM.

Common Bus System

- ★ A common bus system is a technique used to connect multiple registers, memory, and the ALU using a single shared communication pathway, called the bus.
- ★ A basic computer typically consists of 8 registers, a memory unit, an adder and logic circuit, and a control unit, all connected via a single common bus.

Connections in the Bus System

- ★ All register outputs (except OUTR) are connected to the common bus.
- ★ The selection of output to be placed on the bus is done using control signals: S2, S1, and S0 (3-bit binary selector).
- ★ The common bus lines are connected to the inputs of all registers and memory.
- ★ A register loads data from the bus when its LD (Load) signal is activated.
- ★ The memory accepts data from the bus when its Write signal is enabled.
- ★ The memory places data on the bus when its Read signal is activated.



Control Inputs of Registers

Most registers (AR, PC, DR, AC, TR) have three control inputs:

- ★ LD (Load): Load data from the bus
- ★ INR (Increment): Increment register content
- ★ CLR (Clear): Clear register content to 0

These registers behave similarly to binary counters due to the INR control line.

Adder and Logic Circuit

- ★ The Adder and Logic Unit (ALU) provides 16-bit input to the AC (Accumulator).
- ★ It has three input sources:
  1. From AC: For performing register operations
  2. From DR: For executing arithmetic and logic operations
  3. From INPR: For loading data from input devices
- ★ The result of any operation is stored back into the AC.
- ★ E (Extend Flip-Flop): Holds end-around carry during arithmetic operations.

Note:

During a single clock cycle, the contents of any register can be:

★ Placed onto the common bus, and

★ Processed by the adder and logic circuit

★ Then the result is stored back into AC.

INPR and OUTR Behavior:

★ INPR (Input Register): Gets 8-bit data from an input device and passes it to the AC.

★ OUTR (Output Register): Takes 8-bit data from the AC and sends it to the output device.

★ These registers are connected directly to I/O lines and not to the output side of the bus.

रजिस्टर

विषयवस्तु: रजिस्टरों का परिचय, रजिस्टरों की श्रेणियाँ, रजिस्टरों के गुणधर्म (प्रॉपर्टी)।

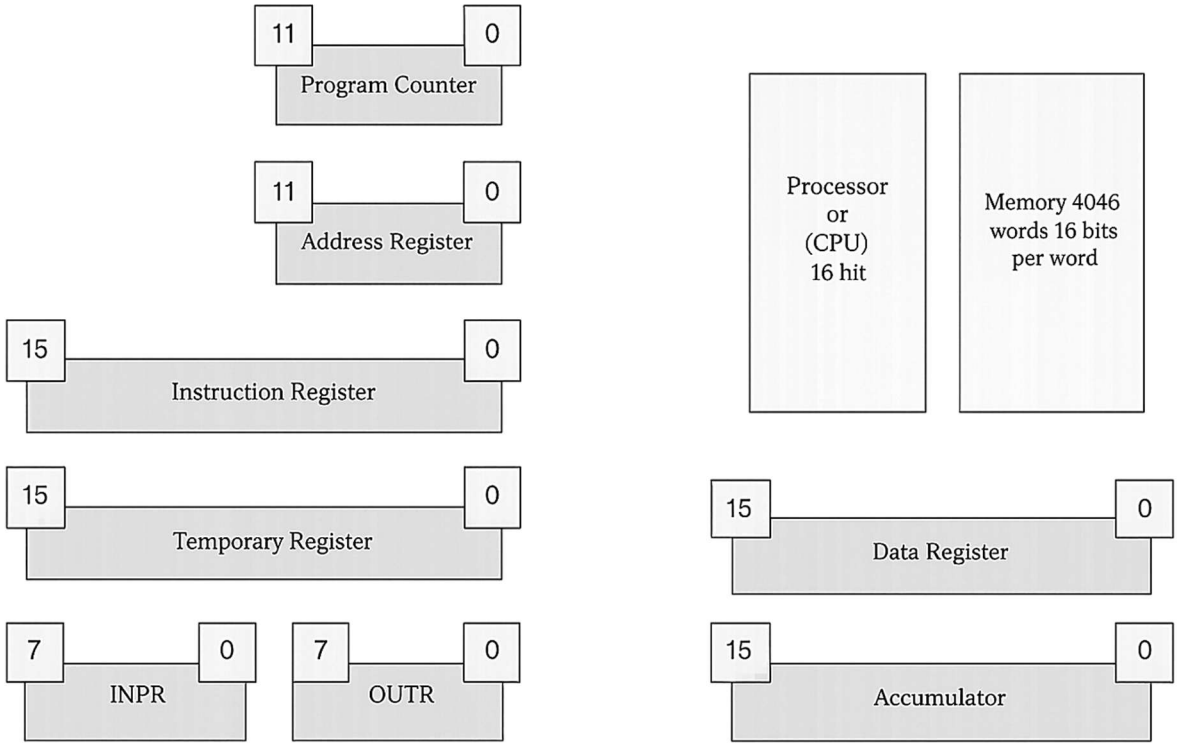
कंप्यूटर में रजिस्टर क्या होता है?

- ★ रजिस्टर CPU के भीतर एक बहुत छोटी और बहुत तेज़ स्टोरेज इकाई है।
- ★ यह CPU के काम करते समय थोड़े समय के लिए डेटा, निर्देश या पते संग्रहीत करता है।
- ★ रजिस्टर कंप्यूटर में सबसे तेज़ प्रकार की मेमोरी है।
- ★ ये CPU को तेजी से कार्य करने में मदद करते हैं क्योंकि CPU इन्हें तुरंत एक्सेस कर सकता है।
- ★ एक रजिस्टर में निम्नलिखित चीजें संग्रहीत की जा सकती हैं:
  - बिट्स (0 और 1)
  - निर्देश
  - मेमोरी अड्रेस
  - तेज़ प्रोसेसिंग के लिए थोड़ी मात्रा में डेटा आवश्यक है

रजिस्टर क्यों महत्वपूर्ण हैं?

- ★ **रजिस्टर, कैश और रैम की तुलना में तेज़ होते हैं,** इसलिए CPU बिना प्रतीक्षा किए तुरंत डेटा प्राप्त कर सकता है।
- ★ **महत्वपूर्ण डेटा CPU** के भीतर ही रहता है, इसलिए ये मेमोरी एक्सेस समय को कम करते हैं।
- ★ **रजिस्टर गणनाओं के** दौरान मध्यवर्ती परिणाम (अस्थायी उत्तर) और अंतिम परिणाम संग्रहीत करते हैं।
- ★ **इनका उपयोग निर्देशों को निष्पादित करने,** डेटा में हेरफेर करने और CPU की स्थिति (जैसे फ्लैग, प्रोग्राम काउंटर आदि) पर नज़र रखने के लिए किया जाता है।

CPU रजिस्टर की श्रेणियाँ/वर्ग:



1. **सामान्य प्रयोजन रजिस्टर (जीपीआर)**
   - ★ प्रतीक: R0, R1, R2, ..., Rn
   - ★ प्रोग्राम निष्पादन के दौरान अस्थायी डेटा, मध्यवर्ती परिणाम और मानों को संग्रहीत करने के लिए उपयोग किया जाता है।
   - ★ असेंबली-स्तरीय निर्देशों द्वारा अभिगमित।
   - ★ रजिस्टर-से-रजिस्टर संक्रियाओं में व्यापक रूप से उपयोग किया जाता है (स्मृति-आधारित संक्रियाओं की तुलना में तेज़)।

2. **संचायक रजिस्टर (AC)**
   - ★ ALU से प्राप्त मध्यवर्ती अंकगणितीय और तार्किक परिणामों को संग्रहीत करता है।
   - ★ यदि निर्देश में स्पष्ट रूप से उल्लेख नहीं किया गया है, तो यह डिफ़ॉल्ट ऑपरेंड/परिणाम स्थान के रूप में कार्य करता है।
   - ★ पुराने या सरल CPU में इसका अक्सर उपयोग किया जाता है।

3. **प्रोग्राम काउंटर (PC)**
   - ★ इसमें निष्पादित होने वाले अगले निर्देश का पता होता है।
   - ★ प्रत्येक फ़ेच चक्र के बाद यह स्वचालित रूप से बढ़ता है।
   - ★ इसका आकार आर्किटेक्चर पर निर्भर करता है (उदाहरण के लिए, एक 12-बिट पीसी 4K मेमोरी स्थानों को संबोधित कर सकता है)।

4. **निर्देश (इन्स्ट्रक्शन) रजिस्टर (IR)**
   - ★ मेमोरी से प्राप्त वर्तमान निर्देश को संग्रहीत करता है।
   - ★ निर्देश को यहीं से डिकोड और निष्पादित किया जाता है।
   - ★ बिटों की संख्या = निर्देश में बिटों की संख्या।

5. **मेमोरी एड्रेस रजिस्टर (MAR)**
   - ★ इसमें उस डेटा या निर्देश का मेमोरी पता होता है जिसे प्राप्त या संग्रहीत किया जाना है।
   - ★ मेमोरी संबंधी कार्यों के लिए MDR के साथ इसका उपयोग किया जाता है।

6. **मेमोरी डेटा रजिस्टर (MDR) / मेमोरी बफर रजिस्टर (MBR)**
   - ★ यह मेमोरी में स्थानांतरित होने वाले/मेमोरी से आने वाले डेटा को अस्थायी रूप से संग्रहीत करता है।
   - ★ यह मेमोरी और CPUके बीच डेटा इंटरफ़ेस के रूप में कार्य करता है।

7. **स्टेक पॉइन्टर (SP)**
   - ★ मेमोरी में स्टैक के शीर्ष को इंगित करता है।
   - ★ स्टैक का उपयोग फ़ंक्शन कॉल, इंटरप्ट और रिकर्सिव प्रक्रियाओं के लिए किया जाता है।
   - ★ SP पुश और पॉप करने पर स्वचालित रूप से अपडेट होता है।

8. **फ्लैग रजिस्टर / कन्डिशन रजिस्टर / कन्डिशन कोड रजिस्टर (CCR)**
   - ★ यह बाइनरी फ़्लैग्स को संग्रहीत करता है जो संचालन के परिणाम का वर्णन करते हैं।.
   - ★ **कॉमन फ्लैग:**
     - ➢ कैरी (C): कैरी/उधार होने पर सेट करें।
     - ➢ जीरो (Z): परिणाम शून्य होने पर सेट करें।
     - ➢ सिम्बल (S) या ऋणात्मक (N): परिणाम ऋणात्मक होने पर सेट करें।
     - ➢ ओवरफ़्लो (V): चिह्नित ओवरफ़्लो होने पर सेट करें।
     - ➢ समता, सहायक कैरी, इंटरप्ट सक्षम करना, आदि।
     - ➢ फ्लैग का उपयोग ब्रांचिंग और सशर्त तर्क में किया जाता है।

9. अस्थायी रजिस्टर (TR)

- ★ जटिल CPU प्रक्रियाओं के दौरान अस्थायी मध्यवर्ती डेटा संग्रहीत करता है।
- ★ प्रोग्रामर द्वारा इसकी पहुँच नहीं है, इसका उपयोग नियंत्रण इकाई द्वारा आंतरिक रूप से किया जाता है।

10. इनपुट रजिस्टर (INPR) और आउटपुट रजिस्टर (OUTR)

- ★ INPR इनपुट डिवाइस से प्राप्त वर्णों को संग्रहित करता है।
- ★ OUTR आउटपुट डिवाइस को भेजे जाने वाले वर्णों को संग्रहित करता है।
- ★ आमतौर पर, 8-बिट रजिस्टर वर्ण एन्कोडिंग पर निर्भर करते हैं।

कॉमन रजिस्टर और उनके गुण

रजिस्टर टेबल

रजिस्टर का नाम	संकेत	बिट्स	कार्य का विवरण
रजिस्टर का नाम	AC	16	अंकगणितीय/तर्क संक्रियाओं का परिणाम भंडारित करता है।
संचायक	PC	12	अगले निर्देश के पता को धारण करता है।
प्रोग्राम गणक	AR	12	डेटा/निर्देश के पता को धारण करता है।
पता रजिस्टर	DR	16	मेमोरी से प्राप्त किए गए ऑपरेंड/डेटा को धारण करता है।
डेटा रजिस्टर	IR	16	वर्तमान में निष्पादित हो रहे निर्देश को भंडारित करता है।
निर्देश रजिस्टर	TR	16	प्रसंस्करण के दौरान अस्थायी मानों को भंडारित करता है।
अस्थायी रजिस्टर	INPR	8	इनपुट डिवाइस से इनपुट कैरेक्टर को धारण करता है।
इनपुट रजिस्टर	OUTR	8	आउटपुट डिवाइस के लिए आउटपुट कैरेक्टर को धारण करता है।

रजिस्टर अन्य CPUघटकों के साथ कैसे काम करते हैं

रजिस्टर इनके साथ संचार करते हैं:

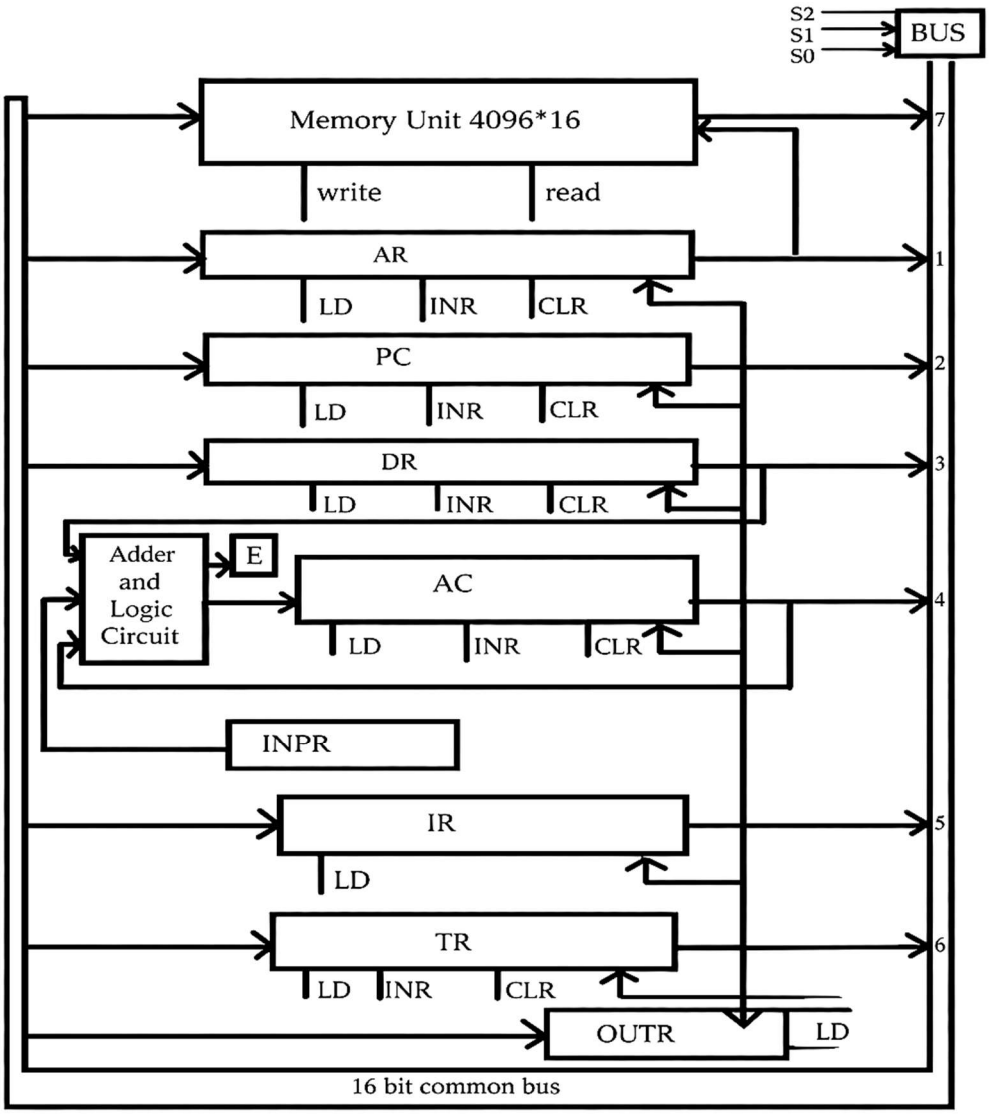
- ★ अंकगणितीय और तार्किक संक्रियाओं के लिए ALU
- ★ निर्देश निष्पादन नियंत्रण के लिए नियंत्रण इकाई
- ★ डेटा/निर्देश प्राप्त करने और संग्रहित करने के लिए मेमोरी
- ★ डेटा स्थानांतरण के लिए I/O उपकरण (INPR/OUTR)
- ★ रजिस्टर तीव्र पहुँच और प्रसंस्करण को सक्षम बनाते हैं, जिससे धीमी RAM पर निर्भरता कम होती है।

कॉमन बस सिस्टम

- ★ कॉमन बस सिस्टम एक ऐसी तकनीक है जिसका उपयोग कई रजिस्टरों, मेमोरी और एएलयू को एक साझा संचार मार्ग (जिसे बस कहते हैं) के माध्यम से जोड़ने के लिए किया जाता है।
- ★ एक सामान्य कंप्यूटर में आमतौर पर 8 रजिस्टर, एक मेमोरी यूनिट, एक एडर और लॉजिक सर्किट, और एक कंट्रोल यूनिट होते हैं, जो सभी एक ही कॉमन बस के माध्यम से जुड़े होते हैं।

कॉमन बस सिस्टम में कनेक्शन

- ★ सभी रजिस्टर आउटपुट (OUTR को छोड़कर) कॉमन बस से जुड़े होते हैं।
- ★ बस पर रखे जाने वाले आउटपुट का चयन कंट्रोल सिग्नल S2, S1 और S0 (3-बिट बाइनरी सेलेक्टर) का उपयोग करके किया जाता है।
- ★ कॉमन बस लाइनें सभी रजिस्टरों और मेमोरी के इनपुट से जुड़ी होती हैं।
- ★ जब रजिस्टर का LD (लोड) सिग्नल सक्रिय होता है, तो वह बस से डेटा लोड करता है।
- ★ जब मेमोरी का राइट सिग्नल सक्रिय होता है, तो वह बस से डेटा स्वीकार करती है।
- ★ जब मेमोरी का रीड सिग्नल सक्रिय होता है, तो वह बस पर डेटा रखती है।



रजिस्टरों के नियंत्रण इनपुट

अधिकांश रजिस्टरों (AR, PC, DR, AC, TR) में तीन नियंत्रण इनपुट होते हैं:

★ LD (लोड): बस से डेटा लोड करें

★ INR (इंक्रीमेंट): रजिस्टर की मात्रा बढ़ाएँ

★ CLR (क्लियर): रजिस्टर की मात्रा को शून्य पर सेट करें

INR कंट्रोल लाइन के कारण ये रजिस्टर बाइनरी काउंटरों के समान व्यवहार करते हैं।

एडर और लॉजिक सर्किट

★ एडर और लॉजिक यूनिट (ALU) AC (एक्युमुलेटर) को 16-बिट इनपुट प्रदान करती है।

★ इसके तीन इनपुट स्रोत हैं:

1. AC से: रजिस्टर संचालन करने के लिए
2. DR से: अंकगणितीय और तार्किक संचालन करने के लिए
3. INPR से: इनपुट उपकरणों से डेटा लोड करने के लिए

★ किसी भी ऑपरेशन का परिणाम AC में संग्रहित किया जाता है।

★ E (एक्सटेंड फ्लिप-फ्लॉप): अंकगणितीय संक्रियाओं के दौरान एंड-अराउंड कैरी को रखता है।

नोट:

एक एकल क्लॉक चक्र के दौरान, किसी भी रजिस्टर की सामग्री निम्न हो सकती है:

★ इसे कॉमन बस पर रखा जाता है, और

★ एडर और लॉजिक सर्किट द्वारा संसाधित किया जाता है।

★ फिर परिणाम को वापस एसी में संग्रहीत किया जाता है।

INPR और OUTR व्यवहार:

★ INPR (इनपुट रजिस्टर): इनपुट डिवाइस से 8-बिट डेटा लेता है और उसे AC को भेजता है।

★ OUTR (आउटपुट रजिस्टर): AC से 8-बिट डेटा लेता है और उसे आउटपुट डिवाइस को भेजता है।

★ ये रजिस्टर सीधे I/O लाइनों से जुड़े होते हैं, न कि बस के आउटपुट साइड से।



CUET PG

DAILY CLASS NOTES

Computer Science

Extra Class Notes

Counters and its Types



Counters and its Types

Types of Counters — Short Explanation with EXAMPLE

A **counter** is a sequential circuit made using flip-flops that goes through a predetermined sequence of states in response to clock pulses.

1. **Asynchronous (Ripple) Counter**
   - Flip-flops do **not receive the clock simultaneously**.
   - Output of one flip-flop triggers the next.
   - Simple but relatively slow because of **propagation delay**.
   - **Example:** Simple frequency divider circuits and basic low-speed counting.

   Example: A basic digital clock circuit can use counters to divide a high-frequency clock into lower frequencies.
2. **Synchronous Counter**
   - All flip-flops receive the **same clock pulse simultaneously**.
   - Faster than an asynchronous counter.
   - **EXAMPLE:** Digital systems where accurate, high-speed counting is required, such as **CPU timing circuits and digital measurement equipment**.
3. **Up Counter**
   - Counts in **ascending order**.
     Example for a 3-bit counter:
     000 → 001 → 010 → 011 → 100 → 101 → 110 → 111
   - **EXAMPLE:** Counting objects on a **production conveyor belt**—each detected product increments the count.
4. **Down Counter**
   - Counts in **descending order**.
     Example:
     111 → 110 → 101 → 100 → 011 → 010 → 001 → 000
   - **EXAMPLE: Countdown timers**, such as a microwave or traffic-light timer.
5. **Up/Down Counter**
   - Can count **both upward and downward**, depending on a control input.
     Example:
     000 → 001 → 010 → 011 *(UP)*
     011 → 010 → 001 → 000 *(DOWN)*
   - **EXAMPLE: Elevator systems**—people entering/leaving can increase or decrease a floor/occupancy count.
6. **Decade Counter / MOD-10 Counter**
   - Has **10 states**, normally from **0 to 9**.
   - After 9, it returns to 0.
     0000 → 0001 → ... → 1001 → 0000
   - **EXAMPLE: Digital clocks, calculators and digital display systems** for decimal counting.
7. **Ring Counter**
   - A shift register in which the output of the last flip-flop is fed back to the first.
   - Usually only **one flip-flop contains 1** at a time.
     Example:
     1000 → 0100 → 0010 → 0001 → 1000
   - **EXAMPLE: LED chasers**, where LEDs turn ON one after another.

8. **Johnson Counter**
   - Also called a **Twisted Ring Counter**.
   - The **complement of the last flip-flop's output** is fed back to the first flip-flop.
   - An (n)-flip-flop Johnson counter has **2n states**.
     For 4 flip-flops → **8 states**.
   - **EXAMPLE: Sequence generation and timing/control circuits**, such as generating multiple non-overlapping timing signals.

9. **Frequency Divider Counter**
   A counter can be used to **divide the frequency** of an input clock.
   For example, a binary counter can divide the clock frequency by powers of 2.
   - **EXAMPLE: Digital clocks and communication systems**, where a high-frequency oscillator is converted into a lower-frequency timing signal.

Quick UGC-NET Revision Table

Counter	Key Feature	Real-Life Example
Asynchronous	Ripple clocking	Low-speed frequency division
Synchronous	Common clock	CPU/timing circuits
Up	Increasing count	Object counter
Down	Decreasing count	Countdown timer
Up/Down	Both directions	Elevator/position counting
Decade (MOD-10)	10 states	Digital clock/display
Ring	One-hot circulating bit	LED chaser
Johnson	Complemented feedback, 2n states	Timing/sequence generation
Frequency Divider	Divides clock frequency	Digital clock

Important for UGC-NET: Remember these three distinctions especially well:
Asynchronous → ripple delay
Synchronous → common clock, faster
Johnson counter with n flip-flops → 2n states



PW Web/App: https://smart.link/7wwosivoicgd4
Library- https://smart.link/sdfez8ejd80if